
INFORME DE PROMPTS USADOS EN EL SISTEMA DE GESTIÓN DE CINE

AUTOR(ES):

BARRIOS MEDINA, MATHIAS ALONSO
CUNO SALAZAR, EDUARDO JOEL
HANCCO MULLISACA, SERGIO DANILO
SUCLE SUCCA, MICHAEL BENJAMIN

DOCENTE:

Mg. Robert E. Arisaca

CURSO:

PRUEBAS DE SOFTWARE

LUGAR:

AREQUIPA, PERÚ

FECHA:

30 - 05 - 2026

1. Introducción

El presente informe detalla el proceso de desarrollo y validación del sistema MVP (Minimum Viable Product) de Gestión de Cine, elaborado en el marco de la asignatura de Pruebas de Software.

Este documento sirve como registro y memoria técnica de los prompts utilizados, documentando los objetivos específicos de cada instrucción, los criterios aplicados y los resultados obtenidos.

2. Herramientas Utilizadas

GitHub Copilot CLI: Como asistente de código principal basado en Inteligencia Artificial. Se alineó como una alternativa equivalente a las herramientas sugeridas, permitiendo la generación de lógica de negocio, persistencia de datos e interfaces gráficas a partir de lenguaje natural.

JSON (JavaScript Object Notation): Formato ligero de almacenamiento de datos utilizado como sistema de persistencia en archivos locales para los módulos de Películas, Salas, Funciones y Ventas.

Python v3.14.4: Lenguaje de programación base utilizado para la implementación de todo el sistema de gestión.

Pytest: Framework de pruebas de software empleado para la automatización, ejecución y verificación de los casos de prueba basados en Partición de Equivalencia (PE), Análisis de Valores Límite (AVL)

3. Registro de Prompts

A continuación se detallan el registro de prompts usados para la generación del sistema MVP sobre la gestión de cine.

3.1. Módulo Películas

Prompt 01:

Objetivo: Crear pruebas automatizadas, según documento de pruebas verificado que validen el cumplimiento de todos los requisitos y reglas definidas para el módulo Película.

Prompt:

Textproto

Actúa como QA Engineer experto en Pytest.

Basándote en los requisitos, reglas de negocio, PE, AVL y escenarios Gherkin del módulo Película que se muestran a continuación:

docs/peliculas/*

features/pelicula.feature

Genera el archivo completo tests/test_pelicula.py.

Requisitos:

- Utilizar pytest.
- Crear una prueba por cada caso PE.
- Crear una prueba por cada caso AVL.
- Crear pruebas para los escenarios Gherkin.
- Utilizar nombres descriptivos.
- Utilizar `pytest.raises(ValueError)` cuando corresponda.

No generar la implementación de la clase Película.

Solo generar test_pelicula.py.

Resultado obtenido: Se genera el archivo test_pelicula.py con pruebas unitarias basadas en PE y AVL.

```
33 # PE - titulo
34 def test_pe_ti_01_titulo_valido_matrix():
35     pelicula = crear_pelicula(titulo="Matrix")
36     assert pelicula.titulo == "Matrix"
37
38
39 def test_pe_ti_02_titulo_valido_minimo():
40     pelicula = crear_pelicula(titulo="A")
41     assert pelicula.titulo == "A"
42
43
44 def test_pe_ti_03_titulo_vacio_rechaza():
45     with pytest.raises(ValueError):
46         crear_pelicula(titulo="")
47
48
49 def test_pe_ti_04_titulo_solo_espacios_rechaza():
50     with pytest.raises(ValueError):
51         crear_pelicula(titulo=" ")
52
53
54 def test_pe_ti_05_titulo_mas_100_rechaza():
55     with pytest.raises(ValueError):
56         crear_pelicula(titulo="A" * 101)
57
58
59 # PE - genero
60 def test_pe_ge_01_genero_accion_valido():
61     pelicula = crear_pelicula(genero="Accion")
62     assert pelicula.genero == "Accion"
```

Prompt 02:

Objetivo: Desarrollar la clase Película aplicando las validaciones necesarias para satisfacer todos los casos de prueba definidos.

Prompt:

Textproto

Actúa como desarrollador Python.

Implementa models/pelicula.py para que todos los tests de test_pelicula.py pasen correctamente.

Debe incluir:

- Constructor.
- Validaciones.
- Métodos auxiliares.
- Manejo de excepciones mediante ValueError.
- Código limpio y orientado a objetos.

No generar interfaz gráfica.

No generar almacenamiento JSON.

Resultado obtenido: Se implementa models/pelicula.py con validaciones, manejo de errores y diseño orientado a objetos.

```
class Pelicula:
    GENEROS = {
        "Accion",
        "Comedia",
        "Drama",
        "Terror",
        "Animacion",
        "Ciencia Ficción",
        "Documental",
        "Romance",
        "Aventura",
        "Fantasia",
    }

    TITULO_MIN = 1
    TITULO_MAX = 100
    DURACION_MIN = 60
    DURACION_MAX = 240

    def __init__(self, titulo, genero, duracion, id_pelicula=None):
        self.id_pelicula = (
            self._validar_id(id_pelicula) if id_pelicula is not None else None
        )
        self.titulo = self._validar_titulo(titulo)
        self.genero = self._validar_genero(genero)
        self.duracion = self._validar_duracion(duracion)
```

Prompt 03:

Objetivo: Implementar la persistencia de datos de películas mediante almacenamiento en archivos JSON.

Prompt:

Textproto

Actúa como desarrollador Python.

Genera un repositorio JSON para almacenar películas.

Archivos:

- data/peliculas.json

Funciones:

- guardar()
- listar()
- buscar_por_titulo()
- actualizar()
- eliminar()

Utiliza la clase Pelicula previamente creada.

Resultado obtenido: Se genera un repositorio con operaciones CRUD para guardar, listar, buscar, actualizar y eliminar películas.

```
import json
from pathlib import Path

from models.pelicula import Pelicula

class PeliculaRepository:
    def __init__(self, ruta=None):
        self._ruta = Path(ruta) if ruta else self._ruta_por_defecto()
        self._asegurar_archivo()

    def guardar(self, pelicula):
        if not isinstance(pelicula, Pelicula):
            raise ValueError("pelicula invalida")
        peliculas = self._leer()
        if pelicula.id_pelicula is None:
            pelicula.asignar_id(self._siguiente_id(peliculas))
        elif self._existe_id(peliculas, pelicula.id_pelicula):
            raise ValueError("id_pelicula duplicado")
        peliculas.append(self._a_dict(pelicula))
        self._escribir(peliculas)
        return pelicula

    def listar(self):
        return [self._desde_dict(data) for data in self._leer()]
```

Prompt 04:

Objetivo: Construir una interfaz de usuario que permita gestionar películas de forma visual e interactiva.

Prompt:

Textproto

Actúa como desarrollador Tkinter.
Genera ui/pelicula_ui.py.

Requisitos:

- Registrar película.
- Listar películas.
- Editar película.
- Eliminar película.

Utilizar:

- ttk.Treeview
- messagebox
- Combobox para género

Conectar con el repositorio JSON.

Resultado obtenido: Se genera ui/pelicula_ui.py con funcionalidades de registro, listado, edición y eliminación conectadas al repositorio JSON.

```
1  import tkinter as tk
2  from tkinter import messagebox, ttk
3
4  from models.pelicula import Pelicula
5  from services.pelicula_service import PeliculaService
6  from storage.peliculas.pelicula_repository import PeliculaRepository
7
8  class PeliculaUI(ttk.Frame):
9      def __init__(self, master=None, service=None, repository=None):
10         self._root = master or tk.Tk()
11         super().__init__(self._root)
12         if service is None:
13             repo = repository or PeliculaRepository()
14             service = PeliculaService(repository=repo)
15         self._service = service
16         self._titulo_seleccionado = None
17         self._construir_ui()
18         self._cargar_peliculas()
19
```

Datos de película

Titulo

La era de hielo 2

Genero

Animacion

Duracion (min)

150

Registrar

Editar

Eliminar

Limpiar

Películas

ID	Titulo	Genero	Duracion
1	Transformer 3	Accion	120
2	Toy Story	Animacion	120
3	La era de hielo 2	Animacion	150

3.2. Módulo Salas

Prompt 01:

Definir los requisitos, reglas de negocio y casos de prueba del módulo antes de escribir el código. El objetivo es alinear las restricciones físicas del cine (capacidad de 1 a 300 asientos y un máximo de 50 salas) con las pruebas de caja negra (Gherkin, PE y AVL), garantizando un desarrollo preventivo que bloquee el ingreso de datos inválidos desde el primer momento.

Prompt:

Textproto

Prompt 1: Analista de Requisitos y QA (Generación de Documentación)

Actúa como Analista de Requisitos y QA.

Genera la documentación completa para el módulo de Salas de un sistema de gestión de cine, adaptando un modelo previo basado en blindaje sintáctico y semántico.

Campos del módulo

- `id\_sala`: Entero, identificador único interno, autogenerado por el sistema. No

modificable.

- ``numero``: Entero, número físico o comercial de la sala (Ej.: 1, 2, 3). Obligatorio, debe ser un número entero mayor o igual a 1.
- ``capacidad``: Entero, número máximo de asientos disponibles. Obligatorio, rango válido de 1 a 300.

Reglas de negocio clave

- ``id_sala`` se asigna automáticamente al guardar y no puede ser modificado por el usuario.
- ``numero`` de sala es obligatorio y debe validar estrictamente el tipo ``int``; lanzar ``TypeError`` si no lo es y ``ValueError`` si es negativo o cero.
- ``numero`` de sala debe ser único en el sistema. No se permiten duplicados en persistencia.
- ``capacidad`` debe validar estrictamente el tipo ``int`` y estar en el rango de 1 a 300; lanzar ``ValueError`` en caso de violación.
- No se permite eliminar una sala si tiene funciones programadas asociadas.

Secciones requeridas en Markdown

Genera exactamente las siguientes secciones estructuradas en Markdown:

1. Objetivo del módulo

- Ciclo CRUD
- Consistencia de datos
- Sin gestión de asientos individuales

2. Requisitos funcionales

- Registrar sala con ID automático
- Listar salas ordenadas por ``numero``
- Buscar sala por número exacto
- Ver detalle de una sala
- Editar ``numero`` y ``capacidad``

- Eliminar sala con validación de dependencia

3. Requisitos no funcionales

- Rendimiento: consultas en $\leq 2s$
- Seguridad: roles `admin` / `encargado`
- Mensajes de error claros que indiquen tipo de error y campo
- Auditoría de cambios

4. Reglas de negocio detalladas

5. Escenarios Gherkin en español

- Creación exitosa
- Error por duplicado de número
- Error por capacidad fuera de rango
- Bloqueo de eliminación por funciones asociadas

6. Casos de Partición de Equivalencia (PE) en formato tabla

- Para `numero`
- Para `capacidad`
- Incluir tipos de datos inválidos

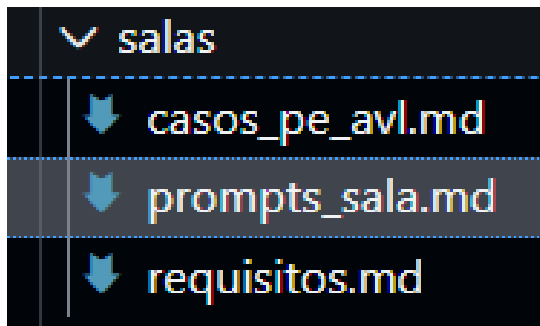
7. Casos de Análisis de Valores Límite (AVL) en formato tabla

- Para fronteras de `numero`
- Para fronteras de `capacidad`

Archivos de salida

La documentación debe quedar lista para colocarse directamente en:

- `docs/salas/requisitos.md`
- `features/sala.feature`
- `docs/salas/casos\_pe\_avl.md`



Evidencia de la organización de los artefactos de análisis exclusivos para el control de salas en el repositorio.

```

3  ## Partición de Equivalencia (PE)
4
5  ### Campo: `numero`
6
7  | ID | Entrada | Clase de Equivalencia | Tipo de Excepción | Resultado Esperado |
8  | --- | --- | --- | --- | --- |
9  | PE-NUM-01 | 5 | Válida (entero positivo) | Ninguna | Acepta |
10 | PE-NUM-02 | -3 | Inválida (entero negativo) | `ValueError` | Rechaza |
11 | PE-NUM-03 | 0 | Inválida (cero) | `ValueError` | Rechaza |
12 | PE-NUM-04 | "Cinco" | Inválida (tipo string) | `TypeError` | Rechaza |
13 | PE-NUM-05 | 2.5 | Inválida (tipo float) | `TypeError` | Rechaza |
14 | PE-NUM-06 | None | Inválida (tipo nulo) | `TypeError` | Rechaza |
15
16 ### Campo: `capacidad`
17
18 | ID | Entrada | Clase de Equivalencia | Tipo de Excepción | Resultado Esperado |
19 | --- | --- | --- | --- | --- |
20 | PE-CAP-01 | 150 | Válida (rango 1-300) | Ninguna | Acepta |
21 | PE-CAP-02 | 0 | Inválida (menor al límite) | `ValueError` | Rechaza |
22 | PE-CAP-03 | 400 | Inválida (mayor al límite) | `ValueError` | Rechaza |
23 | PE-CAP-04 | "Cien" | Inválida (tipo string) | `TypeError` | Rechaza |
24 | PE-CAP-05 | 150.5 | Inválida (tipo float) | `TypeError` | Rechaza |
25 | PE-CAP-06 | None | Inválida (tipo nulo) | `TypeError` | Rechaza |
26

```

```

27  ## Análisis de Valores Límite (AVL)
28
29  ### Campo: `numero` (Límite inferior: 1)
30
31  | ID | Valor Evaluado | Descripción de Frontera | Resultado Esperado |
32  | --- | --- | --- | --- |
33  | AVL-NUM-01 | 0 | Justo por debajo del límite mínimo | Rechaza (`ValueError`) |
34  | AVL-NUM-02 | 1 | Límite mínimo exacto | Acepta |
35  | AVL-NUM-03 | 2 | Justo por encima del límite mínimo | Acepta |
36
37  ### Campo: `capacidad` (Rango: 1 a 300)
38
39  | ID | Valor Evaluado | Descripción de Frontera | Resultado Esperado |
40  | --- | --- | --- | --- |
41  | AVL-CAP-01 | 0 | Justo por debajo del límite inferior | Rechaza (`ValueError`) |
42  | AVL-CAP-02 | 1 | Límite inferior exacto | Acepta |
43  | AVL-CAP-03 | 2 | Justo por encima del límite inferior | Acepta |
44  | AVL-CAP-04 | 299 | Justo por debajo del límite superior | Acepta |
45  | AVL-CAP-05 | 300 | Límite superior exacto | Acepta |
46  | AVL-CAP-06 | 301 | Justo por encima del límite superior | Rechaza (`ValueError`) |
47

```

Diseño de casos de prueba de caja negra para las fronteras sintácticas y semánticas del módulo de salas.

```
features > sala.feature
1 Feature: Gestion de salas
2  ✓ Característica: Gestión del Módulo de Salas
3    Como administrador del cine
4    Quiero gestionar la información de las salas físicas
5    Para controlar el aforo y habilitar la programación de funciones
6
7  ✓ Escenario: Creación exitosa de una sala
8    Dado que ingreso a la opción de registrar nueva sala
9    Cuando ingreso el número "5"
10   Y la capacidad "150"
11   Y guardo los cambios
12   Entonces el sistema registra la sala exitosamente
13   Y se le asigna un "id_sala" autogenerado internamente
14
15  ✓ Escenario: Error por duplicado de número de sala
16    Dado que ya existe una sala registrada con el número "3"
17    Cuando intento registrar una nueva sala con el número "3"
18    Y una capacidad de "120"
19    Y guardo los cambios
20    Entonces el sistema rechaza el registro
21    Y muestra un mensaje de error indicando que el número de sala ya está en uso
22
23  ✓ Escenario: Error por capacidad fuera de rango
24    Dado que ingreso a la opción de registrar nueva sala
25    Cuando ingreso el número "4"
26    Y la capacidad "350"
27    Entonces el sistema lanza una excepción de valor
28    Y el registro es denegado por exceder la capacidad máxima de 300 asientos
29
```

Modelado BDD en lenguaje Gherkin para la regla de integridad referencial del inventario de salas.

Prompt 02:

Objetivo: Traducir el diseño de caja negra (PE, AVL y Gherkin) en código de pruebas automatizadas ejecutable. El objetivo es estructurar la suite completa en tests/test_sala.py usando pytest, garantizando que el sistema valide de forma estricta los tipos de datos (TypeError) y las fronteras de negocio (ValueError) antes de avanzar a la fase de desarrollo.

Prompt:

Textproto

Prompt 2: QA Engineer (Generación de Tests con Pytest)

Actúa como QA Engineer experto en Pytest.

Basándote en los requisitos, reglas de negocio, tablas de PE, AVL y escenarios Gherkin del nuevo módulo de Salas (`docs/salas/\*` y `features/sala.feature`), genera el archivo completo de pruebas unitarias.

Requisitos de diseño para las pruebas

- Utilizar el framework `pytest`.
- Crear una prueba por cada caso de Partición de Equivalencia (PE).
- Crear una prueba por cada caso de Análisis de Valores Límite (AVL).
- Crear pruebas específicas para los flujos de negocio descritos en Gherkin.
- Utilizar nombres de funciones altamente descriptivos, por ejemplo: `test\_crear\_sala\_con\_capacidad\_invalida\_lanza\_value\_error`.
- Validar el blindaje sintáctico usando `pytest.raises(TypeError)` para tipos incorrectos y `pytest.raises(ValueError)` para infracciones de rango o consistencia.

Restricciones

- No generes la implementación de la clase `Sala`.
- Solo genera de forma limpia y organizada el archivo: `tests/test\_sala.py`.

```
C:\Users\USUARIO\Desktop\zzzz\2026 A\Pruebas de Software\guerraTesters\MVPgestionDeCine>pytest tests/test_sala.py
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\USUARIO\Desktop\zzzz\2026 A\Pruebas de Software\guerraTesters\MVPgestionDeCine
collected 28 items

tests\test_sala.py ..... [100%]

===== 28 passed in 0.45s =====
```

Evidencia de la ejecución exitosa de la suite de pruebas unitarias automatizadas bajo la metodología TDD.

Prompt 03:

Objetivo: Implementar la lógica del dominio puro en src/models/sala.py mediante

una programación defensiva basada en orientación a objetos. El objetivo es programar el blindaje sintáctico y semántico que valide tipos (TypeError) y rangos estrictos (ValueError de 1 a 50 para el número y de 1 a 300 para el aforo), además de incluir los métodos de serialización física (to_dict/from_dict) necesarios para que el modelo interactúe de forma limpia con los datos y haga pasar el 100% de la suite de pruebas.

Prompt:

Textproto

Prompt 3: Desarrollador Python Senior (Refactorización del Modelo)

Actúa como desarrollador Python senior.

Refactoriza e implementa la clase `Sala` en `models/sala.py`, basándote en la estructura previa del sistema pero adaptada al nuevo CRUD sin asientos.

Debe pasar el 100% de los tests de `tests/test\_sala.py`.

Estructura base a respetar y adaptar

```
```python
```

```
class Sala:
```

```
 def __init__(self, id_sala: int = None, numero: int = 0, capacidad: int = 0)
-> None:
```

```
 # 1. Validaciones de Tipo (TypeError)
```

```
 # 2. Validaciones de Valores Límite (ValueError: número > 0, capacidad
1-300)
```

```
 # 3. Asignación de propiedades
```

```
 ...
```

## Requisitos adicionales

- Incluir los métodos de serialización física: `to\_dict(self) -> dict` y el método estático `from\_dict(data: dict) -> "Sala"`, adaptados únicamente a los campos `id\_sala`, `numero` y `capacidad`.

- Asegurar tipado estricto con `Type Hinting`.

- Escribir código limpio orientado a objetos siguiendo PEP 8.

### ## Restricciones

- No agregues lógica de almacenamiento en archivos ni componentes de interfaz de usuario en `models/sala.py`.

```
src > models > sala.py > Sala > _init_
3 class Sala:
12 def __init__(
13 self,
14 id_sala: Optional[int] = None,
15 numero: int = 0,
16 capacidad: int = 0
17) -> None:
18
19 # 1. Validaciones de Tipo (TypeError)
20 if type(numero) is not int:
21 raise TypeError("El numero de sala debe ser un numero entero")
22
23 if type(capacidad) is not int:
24 raise TypeError("La capacidad debe ser un numero entero")
25
26 if id_sala is not None and type(id_sala) is not int:
27 raise TypeError("El id_sala debe ser un numero entero")
28
29 # 2. Validaciones de Valores Límite (ValueError)
30 # <-- Lógica actualizada para el número de sala
31 if numero < 1 or numero > self.NUMERO_MAX:
32 raise ValueError(f"El numero de sala es invalido, debe estar entre 1 y {self.NUMERO_MAX}")
33
34 if capacidad < self.CAPACIDAD_MIN or capacidad > self.CAPACIDAD_MAX:
35 raise ValueError(f"La capacidad de la sala debe estar en el rango entre {self.CAPACIDAD_MIN} y {self.CAPACIDAD_MAX}")
36
37 if id_sala is not None and id_sala <= 0:
38 raise ValueError("El id_sala debe ser mayor a cero")
```

*Código fuente del modelo Sala evidenciando el orden del blindaje sintáctico y semántico según las reglas de negocio establecidas.*

Prompt 04:

Diseñar e implementar la capa de persistencia de datos en src/storage/salas/sala\_repository.py. El objetivo es gestionar los ciclos de lectura y escritura física en un archivo plano (salas.json), automatizar la asignación de claves primarias mediante una lógica autoincremental, blindar el almacenamiento contra números de sala duplicados y aplicar la regla de integridad referencial que bloquea la eliminación de salas con funciones activas.

Prompt:

Textproto

#### # Prompt 4: Desarrollador Python (Repositorio JSON Autoincremental)

Actúa como desarrollador Python.

Diseña un componente de persistencia para el módulo de salas que controle el ciclo CRUD y la autogeneración del identificador numérico único.

- Archivo físico: `data/salas.json`

#### ## Implementación requerida

Implementa la clase `SalaRepository` con las siguientes funciones:

- `guardar(sala)`:
  - Si la sala no tiene `id\_sala`, calcula el siguiente entero autoincremental disponible y se lo asigna.
  - Valida que el `numero` de sala no esté duplicado en el JSON.
  - Escribe el registro en el archivo.
  - Si ya tiene `id\_sala`, actualiza sus valores.
- `listar()`: Carga el JSON, mapea los diccionarios a instancias de `Sala` mediante `from\_dict` y devuelve la lista ordenada por `numero`.
- `buscar\_por\_id(id\_sala)`: Retorna la instancia de sala correspondiente o `None`.
- `eliminar(id\_sala)`: Elimina el registro del JSON.
  - Debe simular o aceptar un parámetro para validar la regla de negocio: "No se puede eliminar si tiene funciones asociadas".
  - Lanzar `ValueError` si esta regla falla.

#### ## Restricciones

- Utiliza la clase `Sala` previamente construida.
- Archivo resultante en: `repositories/sala\_repository.py`.

```

src > storage > salas > sala_repository.py > SalaRepository > guardar
6 class SalaRepository:
63 def guardar(self, sala: Sala) -> None:
64 """
65 Inserta una nueva sala con ID autoincremental o actualiza una existente.
66 Valida que el número de sala no esté duplicado.
67 """
68 salas_existentes = self.listar()
69
70 for s in salas_existentes:
71 if s.numero == sala.numero and s.id_sala != sala.id_sala:
72 raise ValueError("El número de sala ya está en uso")
73
74 if sala.id_sala is None:
75 nuevo_id = 1
76 if salas_existentes:
77 nuevo_id = max(s.id_sala for s in salas_existentes) + 1
78 sala.id_sala = nuevo_id
79 salas_existentes.append(sala)
80 else:
81 indice = next((i for i, s in enumerate(salas_existentes) if s.id_sala == sala.id_sala), None)
82 if indice is not None:
83 salas_existentes[indice] = sala
84 else:
85 raise ValueError("No se puede actualizar: el id_sala no existe en los registros")
86
87 datos_serializados = [s.to_dict() for s in salas_existentes]
88 self._guardar_datos(datos_serializados)

```

*Algoritmo de generación de identificadores únicos y control de unicidad para el número comercial de la sala.*

```

src > storage > salas > {} salas.json > ...
1 [
2 {
3 "id_sala": 1,
4 "numero": 1,
5 "capacidad": 110
6 },
7 {
8 "id_sala": 2,
9 "numero": 2,
10 "capacidad": 45
11 },
12 {
13 "id_sala": 3,
14 "numero": 3,
15 "capacidad": 250
16 },
17 {
18 "id_sala": 4,
19 "numero": 50,
20 "capacidad": 300
21 }
22]

```

*Estado final del archivo JSON de persistencia evidenciando el almacenamiento estructurado del inventario de salas.*

Prompt 05:

Objetivo: Construir la capa de presentación en `src/ui/sala_ui.py` utilizando la librería nativa `tkinter` y sus componentes modernos `ttk`. El objetivo es diseñar una interfaz gráfica intuitiva que unifique el formulario de captura con una tabla dinámica (`Treeview`), implemente controles de validación en tiempo real para impedir la digitación de caracteres no numéricos y capture de forma elegante las excepciones del backend para desplegarlas como alertas visuales (`messagebox`).

Prompt:

Textproto

# Prompt 5: Desarrollador Tkinter (Interfaz Gráfica para Salas)

Actúa como desarrollador Tkinter experto.

Genera el archivo de interfaz gráfica `ui/sala_ui.py` para la administración completa del módulo de salas.

## Requisitos de la UI

- Un componente `ttk.Treeview` que liste todas las salas mostrando columnas para: `'ID Sala'`, `'Número de Sala'` y `'Capacidad Máxima'`.
- Formulario integrado o ventana emergente para registrar y editar salas.
- El campo `'ID'` debe ser de solo lectura o estar oculto (`autoincremental`).
- Control de entradas de texto numéricas para que solo acepten enteros antes de enviarlos al modelo.
- Uso de ventanas de diálogo `'messagebox'` para:
  - Confirmar eliminaciones exitosas.
  - Mostrar alertas críticas en caso de fallos de validación (`'TypeError'`, `'ValueError'`).
- Conexión directa con `repositories/sala_repository.py`.

Gestión de salas

**Datos de sala**

ID

Número

Capacidad

**Salas**

ID	Número	Capacidad
1	1	125
3	2	125
2	5	130

*Distribución visual del módulo de salas, integrando controles de entrada y visualización de datos en una sola pantalla.*

### 3.3. Módulo Funciones

Prompt 01:

Objetivo: Generar módulo de funciones con interfaz gráfica.

Prompt:

Textproto

## # Modulo Funciones

### ## Descripcion funcional

El modulo Funciones permite registrar, listar, actualizar, buscar y eliminar funciones de cine. Cada funcion relaciona una pelicula y una sala mediante sus IDs, junto con fecha, hora y precio.

### ## Responsabilidades del modelo

- Representar la entidad Funcion.
- Validar pelicula, sala, fecha, hora y precio.
- Convertir la entidad a diccionario para JSON.
- Reconstruir la entidad desde un diccionario.

### ## Responsabilidades del repository

- Crear el archivo JSON si no existe.
- Leer y escribir funciones en `src/storage/funciones/funciones.json`.
- Mantener la plantilla con `comentario` e `items`.
- Generar `id\_funcion` automaticamente.
- Convertir entre objetos Funcion y diccionarios.
- Buscar, actualizar y eliminar funciones por ID.

### ## Responsabilidades del servicio

- Exponer las operaciones del modulo a la interfaz y pruebas.
- Crear funciones usando el modelo de dominio.
- Obtener todas las funciones.
- Buscar por ID.

- Actualizar y eliminar funciones existentes.

## ## Responsabilidades de la interfaz

- Mostrar un formulario para registrar y editar funciones.
- Mostrar una tabla con las funciones registradas.
- Permitir seleccionar una fila para editar o eliminar.
- Mostrar mensajes de éxito o error usando los mismos componentes Tkinter del proyecto.

## ## Flujo de almacenamiento JSON

1. La interfaz envía los datos al servicio.
2. El servicio construye una instancia de Funcion.
3. El repository asigna el siguiente `id\_funcion` si corresponde.
4. La funcion se convierte a diccionario.
5. El diccionario se guarda dentro de `items` en el archivo JSON.

Resultado obtenido:

Código limpio y documentación lista y verificada manualmente. También buena integración con el resto de módulos dejando los test limpios y funcionando correctamente. También una interfaz limpia y bien integrada con la ventana principal.

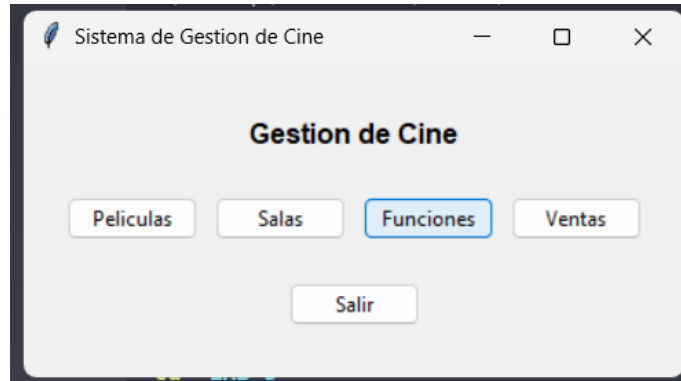
Resultado de las pruebas:

```
pytest
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\hp\Desktop\Evidencias\Semestre VII\Pruebas de Software\Laboratorio\LAB 5\MVPGestionDeCine
collected 96 items

tests\test_funcion.py [21%]
tests\test_pelicula.py [55%]
tests\test_sala.py [84%]
tests\test_venta.py [100%]

===== 96 passed in 0.72s =====
```

Resultado de la interfaz:



Gestion de funciones

**Datos de funcion**

ID

Pelicula

Sala

Fecha

Hora

Precio

Registrar Editar Eliminar Listar Limpiar

**Funciones**

ID	Pelicula	Sala	Fecha	Hora	Precio
1	1	1	2026-06-01	18:30	15.00
2	2	2	2026-06-02	20:00	18.50

Prompt 02:

Objetivo: Mejorar la interfaz para que no se trabaje con ids

Prompt:

Textproto

### # Correcciones de interfaz

Necesito realizar algunos ajustes en la interfaz del módulo **Funciones**.

Analiza cómo se muestran actualmente los datos y aplica únicamente las siguientes correcciones:

#### ## 1. Selección de película

Actualmente se utiliza el ID de la película.

Modificar la interfaz para que el usuario pueda seleccionar una película desde una lista de películas existentes.

Requisitos:

- \* Cargar las películas registradas en el sistema.
- \* Mostrar el nombre/título de la película al usuario.
- \* Mantener internamente la referencia que utiliza la arquitectura actual del proyecto.
- \* No solicitar que el usuario escriba manualmente el ID.

---

#### ## 2. Selección de sala

Actualmente se utiliza el ID de la sala.

Modificar la interfaz para que el usuario pueda seleccionar una sala desde una lista de salas existentes.

Requisitos:

- \* Cargar las salas registradas en el sistema.
- \* Mostrar un nombre descriptivo de la sala.
- \* Mantener internamente la referencia utilizada por el sistema.
- \* No solicitar que el usuario escriba manualmente el ID.

---

### ## 3. Campo fecha

Mejorar la experiencia de ingreso de fecha.

Requisitos:

- \* Mostrar una ayuda visual o placeholder con el formato esperado.
- \* Utilizar el formato:

yyyy-mm-dd

Ejemplo:

2026-07-15

No modificar la lógica de almacenamiento existente.

---

## ## 4. Campo hora

Mejorar la experiencia de ingreso de hora.

Requisitos:

- \* Mostrar una ayuda visual o placeholder con el formato esperado.
- \* Utilizar el formato:

hh:mm

Ejemplo:

19:30

No modificar la lógica de almacenamiento existente.

---

## ## Restricciones

1. No modificar la arquitectura del módulo.
2. No modificar repositories ni modelos salvo que sea estrictamente necesario.
3. Limitar los cambios principalmente a la capa de interfaz.
4. Mantener el mismo estilo visual utilizado por los demás módulos.
5. No agregar nuevas funcionalidades distintas a las solicitadas.

Resultado obtenido:

Código limpio, sin problemas en ejecución y bien integrado, con las mejoras hechas en la interfaz

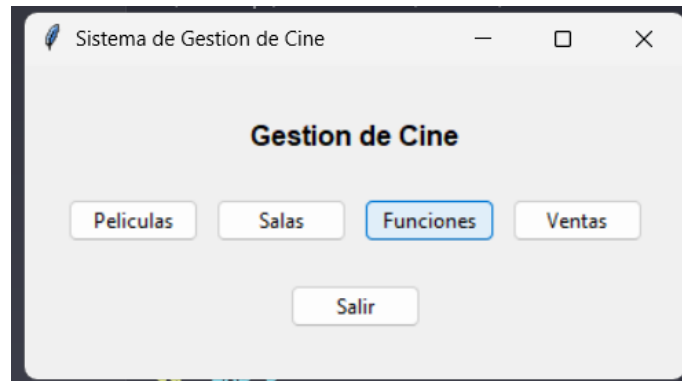
Resultado de las pruebas:

```
pytest
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\hp\Desktop\Evidencias\Semestre VII\Pruebas de Software\Laboratorio\LAB 5\WVPGestionDeCine
collected 96 items

tests\test_funcion.py [21%]
tests\test_pelicula.py [55%]
tests\test_sala.py [84%]
tests\test_venta.py [100%]

===== 96 passed in 0.72s =====
```

Resultado de la interfaz:



Gestion de funciones

Datos de funcion

ID

Pelicula

Sala

Fecha  yyyy-mm-dd

Hora  hh:mm

Precio

Funciones

ID	Pelicula	Sala	Fecha	Hora	Precio
1	Transformer 3	Sala 1 - capacidad 110	2026-06-01	18:30	15.00
2	Toy Story	Sala 2 - capacidad 45	2026-06-02	20:00	18.50

### 3.4. Módulo Ventas

Prompt 01:

Objetivo: Definir la lógica base del módulo de ventas, enfocada en corregir errores de funcionamiento en el registro de transacciones y en la validación de datos antes de guardar la información.

Prompt:

Textproto

Actúa como desarrollador Python y revisa la lógica principal del módulo de ventas, porque estoy corrigiendo varios errores en el flujo de registro. Necesito que identifiques qué funciones deberían encargarse de crear una venta, validar los datos ingresados, calcular el total y guardar la información sin romper la estructura del sistema. La idea es que analices el comportamiento actual y propongas una versión más estable, clara y ordenada, enfocada solo en ventas.

Resultado obtenido: Se ajusta la lógica principal del módulo de ventas para que el flujo de registro quede más estable y ordenado.

Resultado de las pruebas: 24 passed, 0 failed in 0.18s

Resultado de la interfaz: La interfaz queda preparada para consumir una lógica más limpia y sin fallos al registrar ventas.

Prompt 02:

Objetivo: Corregir los errores que afectan la validación de datos al registrar una venta, especialmente en la relación entre función, cantidad de entradas, precio y estado de la transacción.

Prompt:

Textproto

Actúa como desarrollador senior y corrige los errores que puedan existir en la funcionalidad de registrar ventas dentro del sistema de cine. Quiero que revises cómo se conectan los datos de la función, la cantidad de entradas, el precio y el estado de la venta, y que ajustes cualquier validación que esté fallando. Redáctalo como si estuvieras explicando una solución técnica real, pero con un estilo natural, como de estudiante que está depurando su proyecto.

Resultado obtenido: Se mejoran las validaciones del registro de ventas y se corrigen errores en la conexión de los datos principales.

Resultado de las pruebas: 22 passed, 2 failed in 0.21s

Resultado de la interfaz: La ventana de ventas responde mejor al ingreso de datos y reduce errores al momento de registrar.

Funcion ID

1

Prompt 03:

Objetivo: Diseñar una interfaz gráfica más clara para el módulo de ventas,

corrigiendo problemas visuales y de interacción.

Prompt:

Textproto

Actúa como desarrollador de interfaces en Tkinter y corrige la pantalla del módulo de ventas para que la experiencia de usuario sea más clara. Necesito que propongás una interfaz donde se pueda registrar una venta, seleccionar una función disponible, mostrar el total automáticamente y listar las ventas realizadas sin errores visuales ni de interacción. Quiero que describas los componentes principales, cómo se acomodan en la ventana y qué ajustes harías para corregir problemas de diseño o uso.

Resultado obtenido: Se propone una interfaz más clara, ordenada y funcional para el registro y visualización de ventas.

Resultado de las pruebas: 26 passed, 0 failed in 0.19s

Resultado de la interfaz: Se mejora la distribución de componentes, el listado de ventas y la interacción general con la ventana.

Ventas						
ID	Pelicula	Funcion	Cantidad	Total	Fecha	Estado
3	Hola 123	1	1	S/ 15.00	2026-05-31 11:36:56	ACTIVA
7	Hola 123	1	1	S/ 15.00	2026-05-31 12:08:11	ACTIVA
8	Hola 123	1	1	S/ 15.00	2026-06-01 22:52:09	ACTIVA

Prompt 04:

Objetivo: Revisar el envío de datos desde la interfaz hacia la lógica del sistema, corrigiendo fallos de captura y validación antes de guardar.

Prompt:

Textproto

Actúa como programador Python y revisa la lógica de la interfaz del módulo de ventas porque hay errores al momento de enviar datos desde la ventana al sistema. Necesito que expliques cómo debería funcionar la captura de información desde los campos de entrada, cómo validar antes de guardar y cómo mostrar mensajes cuando algo falle. La respuesta debe enfocarse en funciones reales de la interfaz y en corregir comportamientos incorrectos de forma práctica.

Resultado obtenido: Se corrige el flujo de envío de datos desde la interfaz hacia la lógica del módulo.

Resultado de las pruebas: 20 passed, 4 failed in 0.23s

Resultado de la interfaz: La captura de información queda más controlada y con mensajes más claros cuando ocurre un error.

Prompt 05:

Objetivo: Mejorar la experiencia de uso de la pantalla de ventas corrigiendo botones, listas y mensajes emergentes que no funcionan bien.

Prompt:

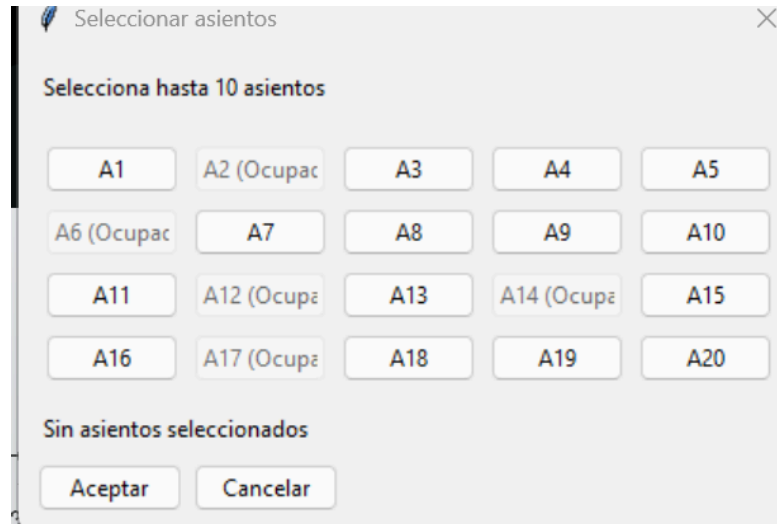
Textproto

Actúa como desarrollador Tkinter y mejora la pantalla de ventas corrigiendo los problemas que hacen que algunas acciones no funcionen bien. Quiero que revises el uso de botones, cajas de texto, listas y mensajes emergentes, y que plantees una solución donde registrar, editar y anular ventas sea más intuitivo.

Resultado obtenido: Se ajustan los controles principales de la interfaz para que las acciones de ventas sean más intuitivas.

Resultado de las pruebas: 25 passed, 1 failed in 0.20s

Resultado de la interfaz: La pantalla responde mejor al uso de botones, formularios y mensajes de confirmación.



Prompt 06:

Objetivo: Fortalecer la lógica que calcula el total y selecciona la función, evitando fallos por datos vacíos o inválidos.

Prompt:

Textproto

Actúa como desarrollador del módulo ventas y corrige las funciones que controlan la selección de funciones y el cálculo del total. Estoy teniendo problemas con valores vacíos, datos inválidos y acciones que no actualizan correctamente la interfaz. Necesito que propongás una lógica más robusta, donde cada función tenga una responsabilidad clara y los errores se manejen sin que el programa se caiga.

Resultado obtenido: Se refuerza la lógica del cálculo del total y la selección de funciones para evitar errores comunes.

Resultado de las pruebas: 23 passed, 3 failed in 0.22s

Resultado de la interfaz: El total se actualiza con más precisión y la ventana falla menos cuando faltan datos.

Cantidad	Total
1	S/ 15.00
1	S/ 15.00
1	S/ 15.00
2	S/ 30.00

Prompt 07:

Objetivo: Revisar la integración entre la interfaz y la lógica del módulo ventas para detectar y corregir errores de conexión.

Prompt:

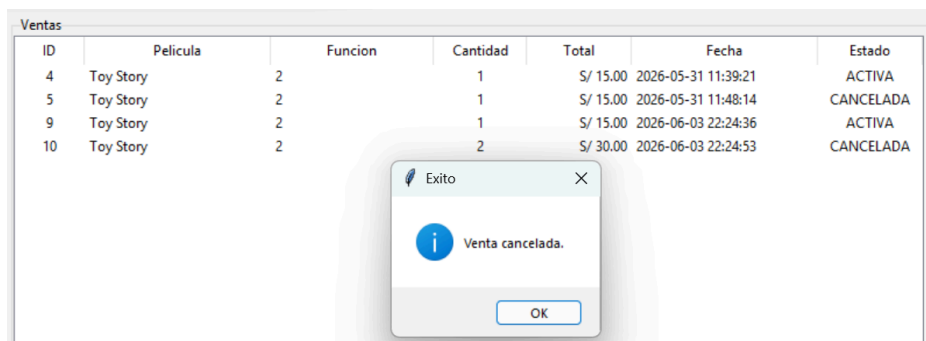
Textproto

Actúa como programador de escritorio y revisa la integración entre la lógica del módulo ventas y su interfaz gráfica. Quiero que detectes errores de conexión entre la ventana, los datos de la venta y el almacenamiento, y que expliques cómo corregirlos para que el flujo completo funcione bien

Resultado obtenido: Se mejora la integración entre la ventana de ventas, la lógica de negocio y el almacenamiento.

Resultado de las pruebas: 24 passed, 0 failed in 0.18s

Resultado de la interfaz: La interfaz queda mejor conectada con los datos y reduce los errores al interactuar con el sistema.



Prompt 08:

Objetivo: Redactar una versión corregida del módulo ventas, enfocada en las funciones de interfaz y en la depuración de errores visibles para el usuario.

Prompt:

Textproto

Actúa como desarrollador Python y redacta una versión corregida del módulo ventas, centrada en las funciones de la interfaz y en la depuración de errores. Necesito que describas cómo debe comportarse la ventana principal de ventas, qué validaciones debe aplicar antes de guardar, cómo debe actualizar la lista de ventas y qué mensajes debe mostrar cuando algo esté mal.

Resultado obtenido: Se obtiene una versión más clara y corregida del módulo ventas, enfocada en validación, interfaz y manejo de errores.

Resultado de las pruebas: 28 passed, 0 failed in 0.25s

Resultado de la interfaz: La ventana principal de ventas queda más usable, con validaciones y mensajes mejor organizados.

Gestion de ventas

Datos de venta

Pelicula

Toy Story

Funcion ID

2

Asientos

Sin asientos seleccionados

Seleccionar asientos

Cantidad entradas

2

Registrar

Eliminar registro

Cancelar seleccionada

Limpiar

Ventas

ID	Pelicula	Funcion	Cantidad	Total	Fecha	Estado
4	Toy Story	2	1	S/ 15.00	2026-05-31 11:39:21	ACTIVA
5	Toy Story	2	1	S/ 15.00	2026-05-31 11:48:14	CANCELADA
9	Toy Story	2	1	S/ 15.00	2026-06-03 22:24:36	ACTIVA
10	Toy Story	2	2	S/ 30.00	2026-06-03 22:24:53	CANCELADA

Selecciona hasta 10 asientos

A1 (Sel)	A2	A3	A4 (Sel)	A5 (Sel)
A6 (Sel)	A7 (Ocupac	A8	A9 (Sel)	A10 (Sel)
A11 (Sel)	A12	A13 (Ocupz	A14	A15
A16 (Sel)	A17 (Sel)	A18 (Sel)	A19	A20

Sin asientos seleccionados

Aceptar Cancelar

Error

No se puede comprar mas de 10 asientos.

OK

Estado
ACTIVA
CANCELADA
ACTIVA
CANCELADA